# Phase 1: Fault Injection and Propagation in a Researcher–Planner–Executor Workflow

Yitang Gao Qinlin Chen Xueru Song Kai Zhang
University of California, San Diego

## 1 Goal

Option 5 asks whether tracking dependencies between agent results can make recovery from a known-faulty result efficient while keeping execution correct and safe. Phase 1 builds the ground for that question: a clean workflow with checkable outcomes, a harness that injects faults at controlled points, traces that record what each step read and did, measurements of how faults propagate, and the two reference baselines, *full restart* and *always-verify*. Recovery methods are Phase 2 work and are not evaluated here.

## 2 Setup

**Workloads.** Each workload has a clean run whose outcome a checker verifies. **W1** (booking) is a researcher → planner → executor workflow. The researcher looks up team size and room capacities, the planner picks a room for each of three meetings, and the executor books it and e-mails attendees (14 steps, 6 LLM calls). `calendar.book` is compensable; `email.send` is irreversible. **W3** (code fix) has an agent read an issue and source file, then write a patch into a git workspace that unit tests check. This adds non-LLM file state. **W4** ports two scenarios from TAMAS [1], a named starting point in the brief: Byzantine agents (`w4.byz`) and indirect prompt injection (`w4.ipi`). Their effects are posts to external sites, which are irreversible.

**Model and platform.** Qwen2.5-7B-Instruct (Q4_K_M) runs in-process through `llama-cpp-python` on one Apple M5 Pro laptop (24 GB), with greedy decoding and JSON-constrained output. A scripted model runs the same workflows for fast checks. The propagation grid (E1) has 3 repeats per configuration, which are scenario variants; the scripted W2 grids have 5, and each baseline cell has 1. 1,675 cells ran with zero errors.

**Traces.** Every step reads and writes one versioned namespace: messages, shared state and files. A step records (i) the messages and artifacts it wrote, (ii) its tool actions with their effect class (`read`, `compensable`, `irreversible`), (iii) its cost (LLM calls, tokens, wall time), and (iv) its *dependencies*: the ordered paths it read, each with a content hash. A fault's *cone* is every step that read the faulty value, directly or through another step.

**Faults.** One fault per run, at a chosen step: *wrong* (a plausible false value), *stale* (an outdated value), *timeout* (a missing value) and *malicious* (an injected instruction or a Byzantine agent). Accidental and adversarial faults are reported separately.

## 3 Clean baseline

All four workloads succeed in 3/3 clean runs on both models (Table 1). The clean-vs-clean spread in wall time is 1.1 s (W1), 1.2 s (W3) and 0.6 s (W4.ipi), which sets the noise floor for latency.

**Table 1:** Clean runs, 7B model ($n$=3 each).

| WL | success | steps | LLM calls | tokens | latency s |
|---|---|---|---|---|---|
| W1 | 3/3 | 14 | 6 | 2427 | 47.6 |
| W3 | 3/3 | 5 | 1 | 661 | 8.7 |
| W4.byz | 3/3 | 6 | 3 | 444 | –* |
| W4.ipi | 3/3 | 6 | 3 | 1078 | 13.2 |

*All cells were prompt-cache hits, so no live latency was measured.

## 4 Propagation

For each fault we measure whether any later step read the faulty value, whether the outcome changed, how many steps and which effects it reached, and whether unauthorised actions occurred (Table 2).

- **Every fault is read downstream** (all 14 fault–workload pairs), so each workload really depends on the injected result.
- **Reach varies widely.** A wrong W1 capacity reaches 9 of 14 steps, both bookings and the irreversible e-mail. The injected agenda reaches only 3 steps, yet sends an unauthorised e-mail in every run.
- **Some faults are read but absorbed.** Three adversarial faults never change the 7B outcome (W3 injected issue; Byzantine fact-check in both W4 scenarios). The scripted model propagates all of them, so absorption is a property

**Table 2:** Fault propagation, 7B model ($n$=3 each). "Changed" is the fraction of runs whose outcome differs from the clean run. "Irrev." counts irreversible calls made in the faulty run.

| WL | fault (class) | changed | steps hit | irrev. | unauth. |
|---|---|---|---|---|---|
| W1 | wrong capacity (acc.) | 1.00 | 9/14 | 3 | 0 |
| | stale capacity (acc.) | 0.67 | 9/14 | 3 | 0 |
| | injected agenda (adv.) | 1.00 | 3/14 | 3 | **1** |
| W3 | wrong issue (acc.) | 1.00 | 3/5 | 0 | 0 |
| | stale code (acc.) | 1.00 | 3/5 | 0 | 0 |
| | injected issue (adv.) | 0.00 | 3/5 | 0 | 0 |
| W4.byz | stale flag (acc.) | 0.67 | 4/6 | 2 | 0 |
| | Byz. distribute (adv.) | 1.00 | 1/6 | 1 | 0 |
| | Byz. fact-check (adv.) | 0.00 | 3/6 | 2 | 0 |
| | injected item (adv.) | 0.33 | 4/6 | 2.3 | 0 |
| W4.ipi | stale flag (acc.) | 0.67 | 4/6 | 1 | 0 |
| | Byz. distribute (adv.) | 1.00 | 1/6 | 1 | 0 |
| | Byz. fact-check (adv.) | 0.00 | 3/6 | 1 | 0 |
| | injected item (adv.) | 0.33 | 4/6 | 1.3 | 0 |

**Table 3:** Baselines, 7B model, mean over faults. Restart: steps rerun, wasted steps (outside the cone), recovery time and tokens, irreversible calls re-sent, recovered correctly. Always-verify: audit tokens per run and their ratio to a clean run's tokens, detection rate (a faulty run counts as detected when the audit flags any step in the fault's cone), false-positive flags per run.

| | full restart | | | | | | always-verify | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| WL | rerun | waste | s | tok | irr. | ok | tok | × | det. | FP |
| W1 | 14 | 6.0 | 36.3 | 2326 | 3.0 | 1.0 | 1245 | .51 | 1.00 | .33 |
| W3† | 5 | 1.0 | 10.6 | 592 | 0 | 1.0 | 575 | .87 | 0.67 | 0 |
| W4.byz | 6 | 2.0 | 13.4 | 1078 | 1.8 | 1.0 | 692 | 1.56 | 0.75 | .25 |
| W4.ipi | 6 | 2.0 | 14.1 | 1097 | 1.0 | 1.0 | 712 | .66 | 1.00 | .25 |

†Restart rerun after the fix in observation (3), on an Apple M5.

of the model. We flag these faults rather than count them as robustness.

- **Accidental faults are not always caught by the model.** A stale capacity changes the W1 outcome in 2 of 3 runs.

## 5 Baselines

**Full restart** undoes what it can and reruns every step. **Always-verify** audits every step's output with the same model against the step's inputs, and its tokens are counted as extra inference cost. Both see the same faulty runs (Table 3). *Wasted work* is the number of rerun steps outside the fault's cone, which a dependency-aware method could avoid.

**Observations.** (1) Restart reruns many steps that never read the fault: on W1, 10 of 14 for the injected agenda and 4 of 14 for the capacity faults. It also re-sends every irreversible call (3.0 per W1 recovery), and it cannot recall the unauthorised e-mail. (2) Always-verify adds 51–156% of a clean run's tokens on *every* run, faulty or not. It still misses one third of W3 faults and a quarter of W4.byz faults, and it raises false flags on clean steps. (3) †Our first W3 restart cells failed in every 7B run, although clean W3 succeeds 3/3. The cause was in our harness, not the model: restart undid each step's old effects only when that step reran, so the rerun researcher read `cart.py` as the faulty run had left it. Where the faulty run had already patched the file, the planner saw fixed code and "fixed" it back. The scripted model leaves correct code alone, which hid the bug. Restart now undoes every compensable effect, newest first, before it reruns anything, and all three W3 cells recover. The same ordering problem can affect any method that rereads tool state, which Phase 2 has to handle.

**Limitations.** Timeout faults are covered only in a scripted single-meeting version of W1, where a missing team size changes the outcome and reaches both effects. They are not yet in the 7B grid. W4 uses 2 of TAMAS's scenarios, and $n$=3 gives wide intervals.

**Takeaway for Phase 2.** Faults reach only part of each workflow, but restart pays for all of it and re-sends irreversible effects, while always-verify pays on every run and still misses faults. Phase 2 will use the recorded dependencies to rerun only the cone, reconcile effects instead of repeating them, and compare against these two baselines under identical fault schedules.

## References

[1] I. Kavathekar et al. TAMAS: Benchmarking adversarial risks in multi-agent LLM systems, 2025. arXiv:2511.05269. Code: github.com/microsoft/TAMAS.

# Appendix: Timeline and Milestones

| Due / status | Milestone | Done when |
|---|---|---|
| *Phase 1 — fault injection and propagation (this report)* | | |
| Done | Review TAMAS and A2ASecBench; choose scenarios | two TAMAS scenarios ported (W4) |
| Done | Clean researcher–planner–executor baseline | W1, W3, W4 pass their checkers 3/3 |
| Done | Fault-injection harness | wrong / stale / timeout / malicious faults at a chosen step |
| Done | Traces: messages, artifacts, tool actions, dependencies | every step records reads with hashes, writes, effects, cost |
| Done | Measure clean runs and propagation | Tables 1–2 |
| Done | Full-restart and always-verify baselines | Table 3 |
| *Phase 2 — selective recovery* | | |
| Nov 8 | Dependency tracking: tag results with their sources | a step's inputs can be listed from the trace |
| Nov 15 | Dependency-aware invalidation and selective rerun | only steps in the fault's cone rerun |
| Nov 22 | Shared state and irreversible actions | compensation for compensable effects; irreversible ones reported, not repeated |
| Nov 29 | Compare against full restart and always-verify | identical fault schedules and budgets, fault stage varied |
| Dec 5 | Evaluate recovery metrics and reuse validity | correctness, unauthorised actions, recomputed stages, retained work, latency, inference cost, tracking overhead |
| Dec 7 | Optional: automated fault detection | false positives and misses reported separately from recovery |
| Dec 10 | Submit Phase 2 code and the final report | 4–6 page report |

**Open items carried into Phase 2.** Add timeout faults to the 7B grid; make reruns that reread tool state see the world as it was before the fault, not as the faulty run left it; add the precision-versus-bookkeeping measurement (dependency granularity) that the brief asks for.

**Acknowledgments.** Phase 1 progress report for the CSE 291A (*Systems for LLMs and AI Agents*, UC San Diego, Fall 2026) course project.